

INDmoney Review Pulse

Turning a manual weekly review-reading chore into an automated, AI-assisted intelligence pipeline

Gauravi Chavan · Solo project, built during PM transition (NextLeap PM Fellowship) · Python, Groq, Gemini, Supabase, GitHub Actions, Streamlit

GitHub: [Click here](#) · Live app: [Click here](#)

Why This Exists

App Store rating isn't just a vanity number — it's a direct install-conversion lever. A declining rating quietly costs new installs before anyone on the team notices, because nobody has time to manually read hundreds of Google Play reviews every week to catch the shift early.

This pipeline exists to close that gap: it reads every review so a person doesn't have to, and pushes a structured summary to the people who'd otherwise be flying blind. It's a small, real tool — two stakeholders are currently subscribed and receive it every Monday.

What It Does

A weekly pipeline runs five phases — ingest → clean → analyze → report → email — fully automated via GitHub Actions:

- Ingest: scrapes Google Play reviews for the INDmoney app within a rolling 12-week window
- Clean: normalizes text and redacts personal information — no LLM involved, pure regex
- Analyze: an LLM clusters the period's reviews into themes, each with representative quotes and a review count
- Report: a second LLM composes the themes into a readable one-pager, with a deterministic fallback if that model call fails
- Email: the report is sent to a configurable recipient list via SMTP, every Monday at 10:00 AM IST

A Streamlit admin app sits alongside the pipeline for managing recipients and reviewing delivery history and past reports — built and iterated on as its own small internal product (see below).

Sample Report — Week of Aug 3, 2026

950 reviews were analyzed in this run (period Jun 8 – Aug 2, 2026) and clustered into five themes. Note the theme labels shift slightly run to run — “Features and Functionality” here versus “Feature Requests / Usability” in earlier runs — a direct example of the open-ended clustering tradeoff discussed below:

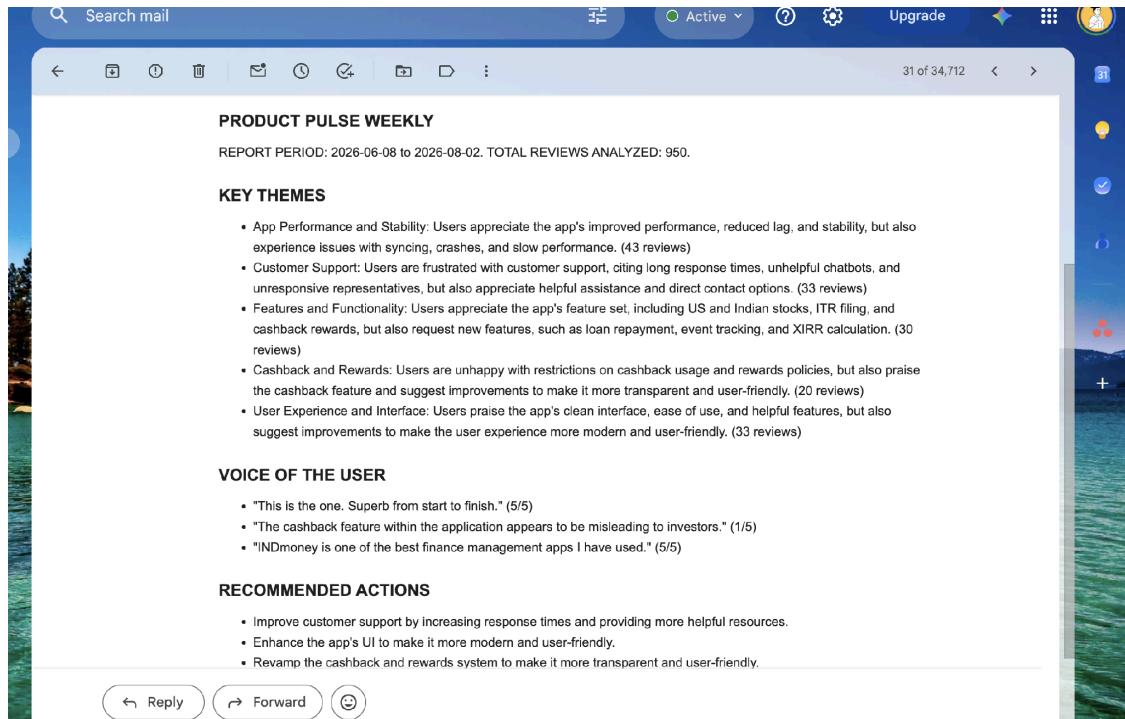
Theme	Reviews
App Performance / Stability	43
Customer Support	33
Features and Functionality	30

User Experience and Interface	33
Cashback / Rewards	20

In the users' own words:

- "This is the one. Superb from start to finish." — 5/5
- "The cashback feature within the application appears to be misleading to investors." — 1/5

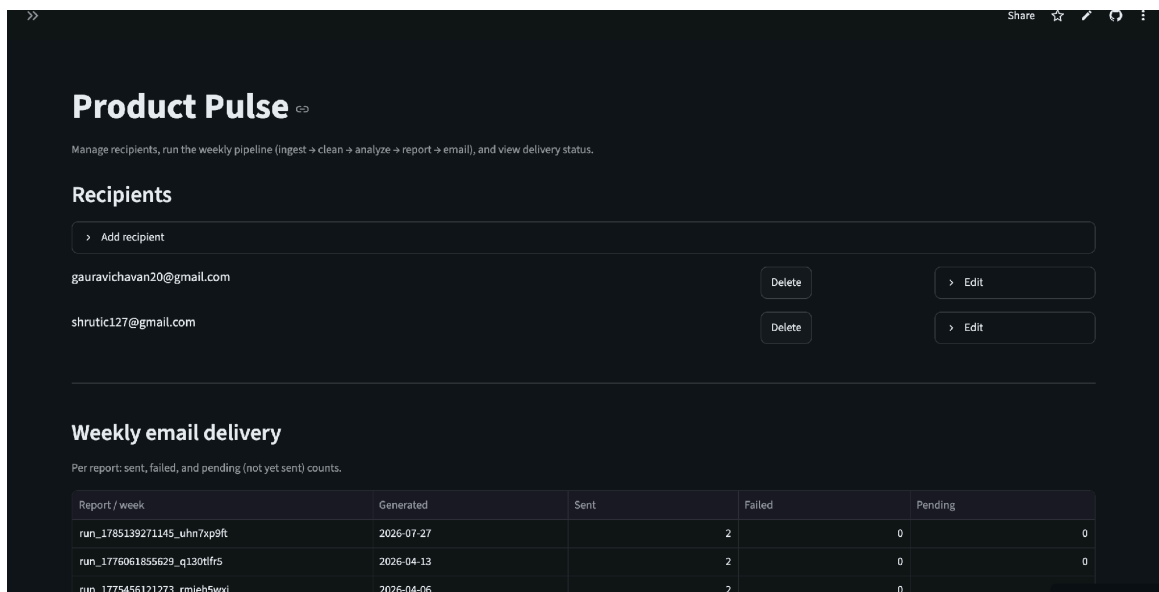
Each theme in the actual report ships with its own quotes and a recommended action — not just a count. Positive and negative signal both surface side by side, so the report doesn't read as purely a complaint log.



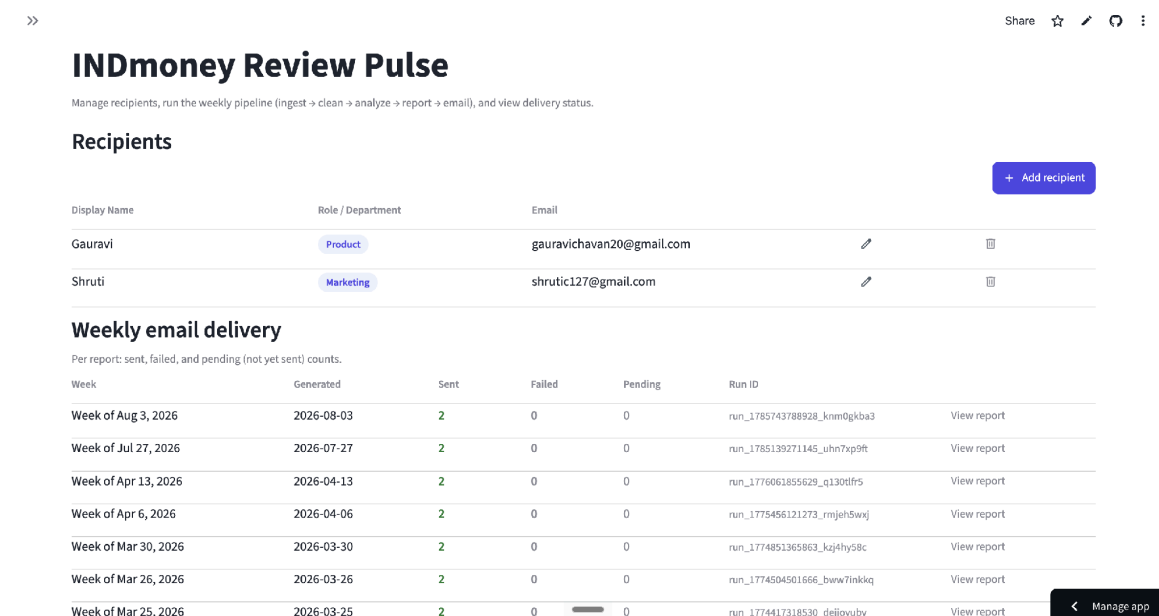
The report as it actually lands — Aug 3, 2026 send, viewed in Gmail

Iterating on the Admin UI

The pipeline worked before the interface did. The first version of the recipient/delivery admin tool was functional but visually generic — default Streamlit styling, raw internal run IDs shown as primary labels, and no visual distinction between a successful and a failed send. As a designer moving into PM, shipping it that way didn't sit right, so I ran it through several rounds of critique and iteration: human-readable labels, color-coded delivery status, a unified table format across sections, and icon-based actions.

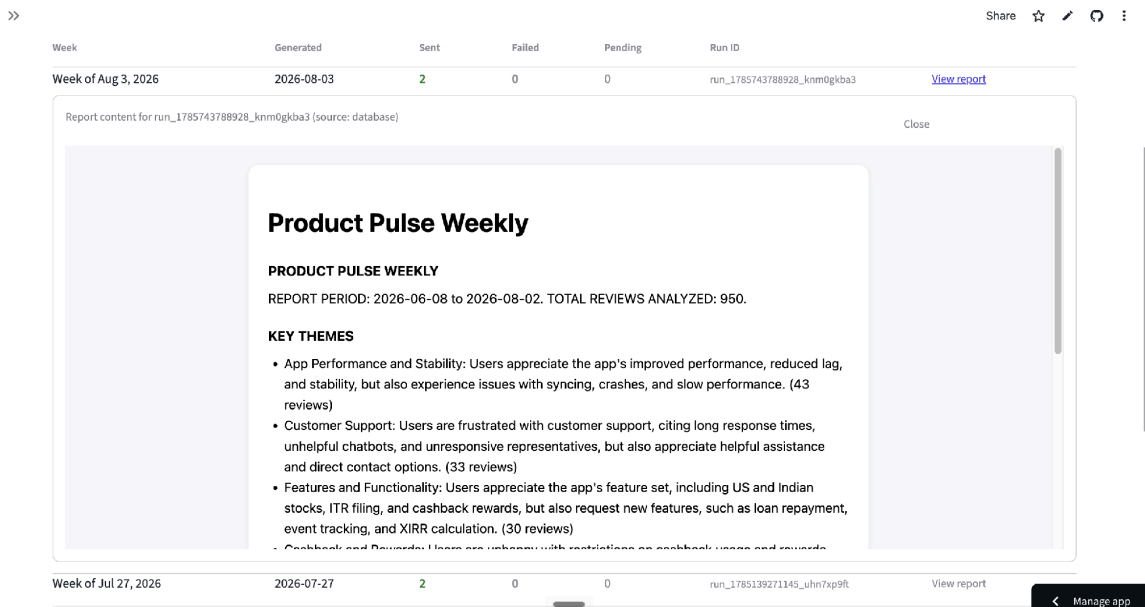


Before — default Streamlit styling, raw run IDs as row labels, no status color-coding



After — human-readable labels, role/department badges, unified table styling, icon actions

One iteration also caught a real backend bug: editing a recipient's role/department field silently failed because the update function's signature hadn't been kept in sync with the create function's. Traceable, fixable, and a good reminder that UI polish passes surface real defects, not just cosmetic ones.



The “View report” panel, including the Close control added after an earlier round left no way to collapse it

PM Thinking — Key Decisions

Two LLM providers, not one

Tension: Phase 3 (theme analysis) fires many calls per run, one per review batch. Phase 4 (report composition) fires one. Routing both through a single provider hit free-tier rate limits.

What I chose: Split providers — Groq for Phase 3, Gemini for Phase 4.

Why: The split was originally forced by hitting limits on one provider, but it turned out to be volume-appropriate on its own merits: a fast, cheap model for the high-call-count step, a stronger model for the single, higher-stakes composition call.

Principle: A constraint can force an architecture that turns out to be correct independent of the constraint — worth keeping even after the original limitation is no longer binding.

Open-ended theme clustering over a fixed taxonomy

Tension: A fixed taxonomy would make themes comparable week over week, which is what trend-tracking needs. But forcing every review into a predefined bucket also means only surfacing problems that fit categories decided in advance.

What I chose: Kept theme labels open-ended — the LLM names themes fresh each run instead of classifying into a fixed list.

Why: A fixed taxonomy risks a “streetlight effect”: you only find what you're structured to look for, and an emerging issue that doesn't fit any bucket goes silently missing or gets mis-filed. That's a worse failure than an unstructured report, because it looks rigorous while quietly being blind to new problems.

Principle: I traded trend-comparability for avoiding measurement bias. Rating-over-time (see What I'd Build Next) is the planned way to get trend signal back without reintroducing that risk.

Email as the primary surface, not the dashboard

Tension: The Streamlit app could have been positioned as where stakeholders go to check the report.

What I chose: Email is the primary delivery surface; the Streamlit app is an admin and exploration tool, not the read surface.

Why: The entire point of this project is removing a manual chore. If consuming the output requires someone to remember to open a dashboard, the chore has just moved, not disappeared.

Principle: Match the delivery mechanism to the behavior you actually want (passive consumption), not the tooling that's easiest to build.

Selective resilience, not uniform resilience

Tension: Every phase could have a fallback to maximize uptime, or some phases can be accepted as hard dependencies.

What I chose: Phase 4 falls back to a deterministic template if Gemini fails. Phase 3 has no fallback for Groq — a failure halts the run.

Why: Phase 4's fallback still produces an honest, if plainer, report. There's no equivalent for Phase 3: a fabricated theme clustering would be actively misleading, which is worse than not sending a report at all.

Principle: Add resilience where the degraded output is still honest. Don't paper over a hard dependency with a weaker approximation just to keep something technically running.

Honest Limitations

- Theme clustering is LLM judgment, not validated against ground truth — no formal accuracy evaluation has been run yet.
- Phase 3 (Groq) has no fallback path — a failure halts the entire run, unlike Phase 4, which degrades gracefully.
- The report period is a rolling window from execution time, not gap-aware — a delayed run doesn't “catch up,” it just looks back the same fixed window from whenever it actually runs.
- The pipeline was deliberately paused for roughly 15 weeks (mid-April to late July 2026) during portfolio and interview prep. Not a reliability failure, but a visible gap in run history worth being upfront about.
- Small recipient list (2) — not yet tested at meaningful scale.
- No trend-over-time tracking on themes, by design (see Key Decisions above) — only raw per-period snapshots currently.

What I'd Build Next

1. Rating-over-time trend

Average star rating per report period, plotted as an interactive chart in the Streamlit dashboard, with a one-line summary added to the email itself since charts don't render reliably in most email clients. This needs no taxonomy — it's a raw number, not an LLM judgment — so it sidesteps the streetlight-effect tradeoff above entirely.

2. A formal eval / validation harness for theme classification

Randomly sample reviews from a run, manually label agree/disagree against the LLM's assigned theme, and track agreement rate plus where disagreements cluster by theme pair — moving theme accuracy from assumed to measured. Planned as its own focused project rather than bolted onto this pipeline, since measurement rigor is a distinct skill worth its own case study.

3. Competitive benchmarking

Apply the same review-analysis pipeline to competitor finance apps (e.g. Zerodha, Wealth Monitor) to distinguish INDmoney-specific issues from industry-wide or seasonal patterns. Sequenced after the above, since it depends on a validated theme model to compare against in the first place.